

Generative AI — Assignment 1

Part 1: Topic Detection & Summarization of News Articles (45 marks)

Dataset: BBC News Archive — 2,225 articles across 5 categories (business, entertainment, politics, sport, tech).

Pipeline built with LangChain:

1. Load dataset into a DataFrame (`df.head(30)`)
2. Topic classification chain (few-shot)
3. Summarization chain (2–3 sentences)
4. Key entity extraction chain (people / organisations / locations, JSON output)
5. Apply all three to every article and merge results back into the DataFrame

The LLM backend is switchable between **Groq** (fast, cloud, rate-limited) and **Ollama** (local, unlimited — use this for the bonus full-dataset run).

0. Setup

```
# Run once if needed
# !pip install -q langchain langchain-core langchain-groq langchain-ollama pandas tqdm python-dotenv

import os, json, re, time, warnings
import pandas as pd
from tqdm.auto import tqdm

warnings.filterwarnings("ignore")
pd.set_option("display.max_colwidth", 120)

# -----
# CHOOSE YOUR BACKEND: "groq" (cloud, fast) or "ollama" (local)
# -----
LLM_PROVIDER = "groq"

# Groq model availability changes over time and can vary by account.
# Set GROQ_MODEL to force a specific model; otherwise select one from the live list.
GROQ_MODEL = os.environ.get("GROQ_MODEL")
OLLAMA_MODEL = "llama3.2:3b" # pull first: ollama pull llama3.2:3b

if LLM_PROVIDER == "groq":
    from groq import Groq
    from langchain_groq import ChatGroq
```

```

    # Prefer capable general-purpose models, then fall back to any
    listed model.
    preferred_models = [
        "openai/gpt-oss-120b",
        "openai/gpt-oss-20b",
        "llama-4-scout-17b-16e-instruct",
        "qwen/qwen3-32b",
        "moonshotai/kimi-k2-instruct",
    ]
    if not os.environ.get("GROQ_API_KEY"):
        from getpass import getpass
        os.environ["GROQ_API_KEY"] = getpass("Enter GROQ_API_KEY: ")

    available_models = {model.id for model in
Groq().models.list().data}
    if GROQ_MODEL:
        if GROQ_MODEL not in available_models:
            raise ValueError(f"GROQ_MODEL={GROQ_MODEL!r} is not
available. Choose from: {sorted(available_models)}")
        else:
            GROQ_MODEL = next((model for model in preferred_models if
model in available_models), None)
            if GROQ_MODEL is None:
                raise RuntimeError(f"No preferred Groq model is available.
Available models: {sorted(available_models)}")

    llm = ChatGroq(model=GROQ_MODEL, temperature=0, max_tokens=512)
else:
    from langchain_ollama import ChatOllama
    llm = ChatOllama(model=OLLAMA_MODEL, temperature=0,
num_predict=512)

print("Backend:", LLM_PROVIDER, "| model:", GROQ_MODEL if LLM_PROVIDER
== "groq" else OLLAMA_MODEL)

Backend: groq | model: openai/gpt-oss-120b

```

Step 1: Load the Dataset

The file is **tab-separated** (despite the `.csv` extension), with columns `category`, `filename`, `title`, `content`.

```

CSV_PATH = "bbc-news-data.csv"    # adjust path if needed

df_full = pd.read_csv(CSV_PATH, sep="\t")
df_full.insert(0, "Article_ID", df_full.index)           # stable ID

```

```

for every article
df_full = df_full.rename(columns={
    "category": "True_Category",
    "title": "Title",
    "content": "Article_Text",
})
df_full["Article_Text"] = df_full["Article_Text"].str.strip()

print("Full dataset shape:", df_full.shape)
display(df_full["True_Category"].value_counts())
df_full.head(3)

```

Full dataset shape: (2225, 5)

```

True_Category
sport          511
business       510
politics       417
tech           401
entertainment  386
Name: count, dtype: int64

```

	Article_ID	True_Category	filename	Title \
0	0	business	001.txt	Ad sales boost Time Warner profit
1	1	business	002.txt	Dollar gains on Greenspan speech
2	2	business	003.txt	Yukos unit buyer faces loan claim

```

Article_Text
0 Quarterly profits at US media giant TimeWarner jumped 76% to $1.13bn (£600m) for the three months to December, from ...
1 The dollar has hit its highest level against the euro in almost three months after the Federal Reserve head said the...
2 The owners of embattled Russian oil giant Yukos are to ask the buyer of its former production unit to pay back a $90...

```

Assignment requirement: limit to the first 30 articles

```
df = df_full.head(30).copy()
```

```

print("Working subset:", df.shape)
display(df[["Article_ID", "True_Category", "Title"]].head(10))

```

NOTE: the file is sorted by category, so head(30) is entirely 'business'.

For a more informative accuracy check, a stratified sample is also prepared below (optional).

```
df_stratified = (df_full.groupby("True_Category", group_keys=False)
```

```

category = 30 articles
        .apply(lambda g: g.head(6))           # 6 per
        .reset_index(drop=True))
print("\nOptional stratified subset:", df_stratified.shape)
df_stratified["True_Category"].value_counts()

```

Working subset: (30, 5)

	Article_ID	True_Category	Title
0	0	business	Ad sales boost Time Warner profit
1	1	business	Dollar gains on Greenspan speech
2	2	business	Yukos unit buyer faces loan claim
3	3	business	High fuel prices hit BA's profits
4	4	business	Pernod takeover talk lifts Domecq
5	5	business	Japan narrowly escapes recession
6	6	business	Jobs growth still slow in the US
7	7	business	India calls for fair trade rules
8	8	business	Ethiopia's crop production up 24%
9	9	business	Court rejects \$280bn tobacco case

Optional stratified subset: (30, 5)

```

True_Category
business      6
entertainment 6
politics      6
sport         6
tech          6
Name: count, dtype: int64

```

Step 2: Topic Classification Task (10 marks)

A **few-shot prompt** that forces the model to answer with exactly one label out of **Business**, **Entertainment**, **Politics**, **Sport**, **Tech**. A small post-processing function snaps any stray output back onto the allowed label set.

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

CATEGORIES = ["Business", "Entertainment", "Politics", "Sport",
              "Tech"]

classification_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a precise news topic classifier. You reply with ONE word only: "
     "Business, Entertainment, Politics, Sport or Tech. No

```

```

explanation, no punctuation."),

    # --- few-shot examples ---
    ("human", "Article:\nShares in the bank rose 4% after it reported
record annual pre-tax profits "
              "and raised its dividend for shareholders.\nCategory:"),
    ("ai", "Business"),

    ("human", "Article:\nThe actress collected the best supporting
role award at Sunday's ceremony, "
              "capping a strong year for the British film industry.\n
Category:"),
    ("ai", "Entertainment"),

    ("human", "Article:\nThe home secretary defended the new anti-
terror bill in the Commons as MPs "
              "prepared to vote on the controversial detention
powers.\nCategory:"),
    ("ai", "Politics"),

    ("human", "Article:\nThe defending champion eased through to the
quarter-finals in straight sets "
              "after his opponent retired injured.\nCategory:"),
    ("ai", "Sport"),

    ("human", "Article:\nThe firm released a security patch for its
browser after researchers "
              "demonstrated a flaw that could let attackers run
malicious code.\nCategory:"),
    ("ai", "Tech"),

    # --- actual task ---
    ("human", "Analyze the following news article and identify its
topic as one of the following "
              "categories: Business, Entertainment, Politics, Sport,
or Tech.\n\n"
              "Title: {title}\nArticle:\n{article}\n\nCategory:"),
    ])

classification_chain = classification_prompt | llm | StrOutputParser()

def normalise_category(raw: str) -> str:
    """Snap free-form model output onto the allowed label set."""
    text = (raw or "").strip().lower()
    for c in CATEGORIES:
        if re.search(rf"\b{c.lower()}\b", text):
            return c
    aliases = {"technology": "Tech", "sports": "Sport", "finance":
"Business", "economy": "Business"}

```

```

    for k, v in aliases.items():
        if k in text:
            return v
    return "Other"

def normalise_category(raw: str) -> str:
    text = re.sub(r"<think>.*?</think>", "", raw or "")
    flags=re.DOTALL).strip().lower()
    for c in CATEGORIES:
        if re.search(rf"\b{c.lower()}\b", text):
            return c
    aliases = {"technology": "Tech", "sports": "Sport", "finance":
"Business", "economy": "Business"}
    for k, v in aliases.items():
        if k in text:
            return v
    return "Other"

```

Step 3: Summarization Task (10 marks)

A prompt that asks for a factual 2–3 sentence summary covering who / what / when / where / why, with no personal commentary.

```

summary_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a professional news editor. You write factual, neutral summaries. "
     "Never add opinion, speculation or commentary. Never start with phrases like "
     "'This article' or 'Here is a summary' – just write the summary."),
    ("human",
     "Summarize the main points of the following news article in 2-3 sentences. "
     "Cover who, what, when, where and why where applicable.\n\n"
     "Title: {title}\nArticle:\n{article}\n\nSummary:")),
])

summarization_chain = summary_prompt | llm | StrOutputParser()

sample = df.iloc[0]

def strip_reasoning(text) -> str:
    if not isinstance(text, str):
        text = getattr(text, "content", "") or str(text)
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL)
    text = re.sub(r"^\``(?:\w+)?|\``$", "", text.strip()),

```

```

flags=re.MULTILINE)
    return text.strip()

summary = strip_reasoning(summarization_chain.invoke({
    "title": sample["Title"],
    "article": sample["Article_Text"][:6000],
}))

print("Title:", sample["Title"], "\n")
print("Summary:\n", summary)

```

Title: Ad sales boost Time Warner profit

Summary:

Time Warner reported a 76% rise in fourth-quarter profit to \$1.13 billion for the three months ended December, driven by higher sales of high-speed internet connections, stronger advertising revenue and one-off gains that offset a dip at Warner Bros. The company, which now holds an 8% stake in Google, said its AOL unit lost 464,000 subscribers but saw underlying profit rise 8% as it pushes free AOL service to its broadband customers, while its film division's profit fell 27% after box-office failures. Time Warner also announced it will restate its 2000 and 2003 results following an SEC investigation, posted a full-year profit of \$3.36 billion up 27%, and projected about 5% operating-earnings growth for 2005.

Step 4: Key Entity Extraction (10 marks)

The model returns **structured JSON** with three buckets — people, organizations, locations — validated with a Pydantic parser. A regex fallback recovers the JSON if the model wraps it in prose or code fences (common with small local models).

```

from pydantic import BaseModel, Field
from typing import List
from langchain_core.output_parsers import JsonOutputParser

class Entities(BaseModel):
    """Identify people, organizations, and locations"""
    people: List[str] = Field(
        description="Names of notable people"
    )
    organizations: List[str] = Field(
        description="Companies, institutions, teams, parties"
    )
    locations: List[str] = Field(
        description="Countries, cities, regions, venues"
    )

entity_parser = JsonOutputParser(pydantic_object=Entities)

```

```

entity_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are an expert named-entity extraction system. You output  

ONLY valid JSON – "  

     "no markdown fences, no commentary. Use the exact spelling from  

the article and do not "  

     "invent entities. If a category has no entities, return an empty  

list."),
    ("human",
     "From the article below, list the names of any important people,  

organizations and places "  

     "mentioned.\n\n{format_instructions}\n\nTitle: {title}\nArticle:\n{n{article}}\n\nJSON:"),
]).partial(format_instructions=entity_parser.get_format_instructions()
)

```

```

entity_chain = entity_prompt | llm | StrOutputParser()

```

```

def parse_entities(raw: str) -> dict:
    """Robustly turn the model's reply into {people, organizations,
locations}."""
    empty = {"people": [], "organizations": [], "locations": []}
    if not raw:
        return empty
    text = re.sub(r"```(?:json)?|`\"", "", raw).strip()
    match = re.search(r"\{.*\}", text, re.DOTALL) # grab the
first JSON object
    if not match:
        return empty
    try:
        data = json.loads(match.group(0))
    except json.JSONDecodeError:
        return empty
    out = {}
    for key in empty:
        val = data.get(key, [])
        if isinstance(val, str):
            val = [v.strip() for v in val.split(",") if v.strip()]
        out[key] = [str(v).strip() for v in val if str(v).strip()]
    return out

```

```

def flatten_entities(ent: dict) -> list:
    """Single de-duplicated list, as in the assignment's example
JSON."""
    seen, flat = set(), []
    for v in ent["people"] + ent["organizations"] + ent["locations"]:
        if v.lower() not in seen:

```

```

        seen.add(v.lower())
        flat.append(v)
    return flat

# ---- Sample datapoint (Expected Output for Step 4) ----
raw_ent = entity_chain.invoke({"title": sample["Title"],
                              "article": sample["Article_Text"]
                              [:6000]})
entities = parse_entities(raw_ent)

print("Raw output:\n", raw_ent.strip()[:600], "\n")
print(json.dumps(entities, indent=2))
print("\nFlattened Key_Entities:", flatten_entities(entities))

```

Raw output:

```

{
  "people": [],
  "organizations": [],
  "locations": []
}

```

Flattened Key_Entities: []

Step 5: Apply to Every Article & Update the DataFrame (15 marks)

Each article runs through all three chains. `safe_invoke` adds retry-with-backoff so a single rate-limit hit doesn't kill the whole run, and `TEXT_LIMIT` truncates very long articles to keep inference fast.

```

TEXT_LIMIT = 6000      # characters fed to the LLM
MAX_RETRIES = 3
SLEEP      = 0.0       # increase (e.g. 1.5) if you hit Groq rate
limits

def safe_invoke(chain, payload, default=""):
    for attempt in range(MAX_RETRIES):
        try:
            return chain.invoke(payload)
        except Exception as e:
            wait = 2 ** attempt
            print(f" ! {type(e).__name__}: {str(e)[:90]} - retry in
{wait}s")
            time.sleep(wait)

```

```

    return default

def process_article(row: pd.Series) -> dict:
    payload = {"title": row["Title"], "article": row["Article_Text"]
[:TEXT_LIMIT]}

    topic_raw = safe_invoke(classification_chain, payload)
    summary = safe_invoke(summarization_chain, payload)
    ent_raw = safe_invoke(entity_chain, payload)
    entities = parse_entities(ent_raw)

    return {
        "Article_ID": row["Article_ID"],
        "Detected_Topic": normalise_category(topic_raw),
        "Summary": summary.strip(),
        "Key_Entities": flatten_entities(entities),
        "People": entities["people"],
        "Organizations": entities["organizations"],
        "Locations": entities["locations"],
    }

def run_pipeline(frame: pd.DataFrame) -> pd.DataFrame:
    records = []
    for _, row in tqdm(frame.iterrows(), total=len(frame),
desc="Processing articles"):
        records.append(process_article(row))
        if SLEEP:
            time.sleep(SLEEP)
    return pd.DataFrame(records)

results_df = run_pipeline(df) # <- the 30-article run
print("Results shape:", results_df.shape)
results_df.head()

{"model_id": "8394c434821b40d6aca75fdd8a0eb5c1", "version_major": 2, "version_minor": 0}

```

```

! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 1s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 1s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s

```

[illegible]

```
llama-3.1-8b-instant` does not exist o – retry in 1s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 2s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 4s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 1s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 2s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 4s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 1s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 2s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 4s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 1s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 2s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 4s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 1s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 2s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 4s  
! NotFoundError: Error code: 404 - {'error': {'message': 'The model  
llama-3.1-8b-instant` does not exist o – retry in 1s
```

[illegible]

```

`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 1s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 1s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 1s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 2s
! NotFoundError: Error code: 404 - {'error': {'message': 'The model
`llama-3.1-8b-instant` does not exist o – retry in 4s
Results shape: (30, 7)

```

	Article_ID	Detected_Topic \
0	0	Other
1	1	Other
2	2	Other
3	3	Other
4	4	Other

Summary \

- 0 Time Warner reported a 76% rise in fourth-quarter profit to \$1.13 billion for the three months ended December, drive...
- 1 The dollar rose to \$1.2871 per euro, its strongest level in nearly three months, after Federal Reserve Chairman Alan...
- 2 Menatep Group, the owner of the former Yukos production unit Yugansk, will demand that state-owned Rosneft repay a \$...
- 3 British Airways reported a 40 % fall in pre-tax profit to £75 million for the three months to 31 December 2004, attr...
- 4 Allied Domecq's London-listed shares rose about 4% after Wall Street Journal and Financial Times reports suggested F...

	Key_Entities	People	Organizations	Locations
0	[]	[]	[]	[]
1	[]	[]	[]	[]
2	[]	[]	[]	[]
3	[]	[]	[]	[]
4	[]	[]	[]	[]

```
# ---- Run the pipeline on the 30-article subset ----
def run_pipeline(frame: pd.DataFrame) -> pd.DataFrame:
    records = []
    for _, row in tqdm(frame.iterrows(), total=len(frame),
desc="Processing articles"):
        records.append(process_article(row))
        if SLEEP:
            time.sleep(SLEEP)
    return pd.DataFrame(records)

results_df = run_pipeline(df)
print("Results shape:", results_df.shape)
results_df.head()

{"model_id": "c85cb5dc2a73455d89e1a854599c5ee9", "version_major": 2, "version_minor": 0}
```

```
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b` - retry in 1s
```

```

! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 4s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 1s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 2s
! RateLimitError: Error code: 429 - {'error': {'message': 'Rate
limit reached for model `openai/gpt-oss-120b - retry in 4s
Results shape: (30, 7)

```

	Article_ID	Detected_Topic	\
0	0	Business	
1	1	Business	
2	2	Business	
3	3	Business	
4	4	Business	

```

Summary \
0 Time Warner reported a 76% rise in fourth-quarter profit to
$1.13 billion for the three months ended December, drive...
1 The U.S. dollar rose to its strongest level against the euro in
nearly three months, reaching $1.2871 in late New Yo...
2 Menatep Group, the owner of the former Yukos production unit
Yugansk, says it will demand that state-owned Rosneft r...
3 British Airways reported a 40% drop in pre-tax profit to
£75 million for the three months to 31 December 2004, blami...
4 Allied Domecq's London-listed shares rose about 4% after Wall
Street Journal and Financial Times reports suggested F...

```

	Key_Entities	People	Organizations	Locations
0	[]	[]		[]
1	[]	[]		[]
2	[]	[]		[]
3	[]	[]		[]
4	[]	[]		[]

```

# ---- Full record view, matching the example JSON in the assignment
----
if "final_df" not in globals():
    df["Article_ID"] = df["Article_ID"].astype(int)
    results_df["Article_ID"] = results_df["Article_ID"].astype(int)

```

```

final_df = df.merge(results_df, on="Article_ID", how="left")[[
    "Article_ID", "Title", "Article_Text", "True_Category",
    "Detected_Topic", "Summary", "Key_Entities",
    "People", "Organizations", "Locations",
]]

rec = final_df.iloc[0].to_dict()
rec["Article_ID"] = int(rec["Article_ID"])
rec["Article_Text"] = rec["Article_Text"][:200] + " ... [excerpt]"
print(json.dumps(rec, indent=2, ensure_ascii=False))

{
  "Article_ID": 0,
  "Title": "Ad sales boost Time Warner profit",
  "Article_Text": "Quarterly profits at US media giant TimeWarner
jumped 76% to $1.13bn (£600m) for the three months to December, from
$639m year-earlier. The firm, which is now one of the biggest
investors in Google, ... [excerpt]",
  "True_Category": "business",
  "Detected_Topic": "Business",
  "Summary": "Time Warner reported a 76% rise in fourth-quarter profit
to $1.13 billion for the three months ended December, driven by higher
sales of high-speed internet connections, stronger advertising revenue
and one-off gains that offset a dip at Warner Bros. The company, which
now holds an 8% stake in Google, said its AOL unit lost 464,000
subscribers but saw underlying profit rise 8% on ad growth, and it
plans to boost subscribers by offering AOL free to Time Warner
internet customers while it restates 2000-2003 results after an SEC
probe. For the full year, Time Warner's profit grew 27% to
$3.36 billion on a 6.4% revenue increase, though its film division saw
a 27% profit decline and the firm is negotiating a $300 million
settlement related to the AOL investigation.",
  "Key_Entities": [],
  "People": [],
  "Organizations": [],
  "Locations": []
}

```

Sanity check — classification accuracy against the dataset's own labels

```

final_df["Correct"] = (final_df["Detected_Topic"].str.lower()
                       == final_df["True_Category"].str.lower())

acc = final_df["Correct"].mean()
print(f"Accuracy on {len(final_df)} articles: {acc:.1%}")

display(pd.crosstab(final_df["True_Category"],
                    final_df["Detected_Topic"]))

misses = final_df.loc[~final_df["Correct"], ["Title", "True_Category",

```

```
"Detected_Topic"]]
if len(misses):
    print("\nMisclassified:")
    display(misses)
```

Accuracy on 30 articles: 80.0%

Detected_Topic	Business	Other	Politics
True_Category			
business	24	2	4

Misclassified:

	Title	True_Category	Detected_Topic
7	India calls for fair trade rules	business	Politics
9	Court rejects \$280bn tobacco case	business	Politics
11	Indonesians face fuel price rise	business	Politics
14	Air passengers win new EU rights	business	Politics
28	UK firm faces Venezuelan land row	business	Other
29	Soaring oil 'hits world economy'	business	Other

```
# ---- Save outputs ----
```

```
final_df.to_csv("part1_bbc_results_30.csv", index=False)
```

```
with open("part1_bbc_results_30.json", "w", encoding="utf-8") as f:
```

```
    json.dump(final_df.drop(columns=["Correct"]).to_dict(orient="records"),
        f, indent=2, ensure_ascii=False)
```

```
print("Saved: part1_bbc_results_30.csv / .json")
```

Saved: part1_bbc_results_30.csv / .json

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field
from typing import List
```

```
# ----- dataset -----
```

```
if "df_full" not in globals():
    df_full = pd.read_csv("bbc-news-data.csv", sep="\t")
    df_full.insert(0, "Article_ID", df_full.index)
    df_full = df_full.rename(columns={"category": "True_Category",
                                    "title": "Title",
                                    "content": "Article_Text"})
    df_full["Article_Text"] = df_full["Article_Text"].str.strip()
```

```
# ----- categories + normaliser -----
```

```
CATEGORIES = ["Business", "Entertainment", "Politics", "Sport",
```

```

    "Tech"]
VALID_TOPICS = set(CATEGORIES)

def normalise_category(raw: str) -> str:
    text = re.sub(r"<think>.*?</think>", "", raw or "")
    flags=re.DOTALL).strip().lower()
    for c in CATEGORIES:
        if re.search(rf"\b{c.lower()}\b", text):
            return c
    for k, v in {"technology": "Tech", "sports": "Sport",
                "finance": "Business", "economy":
"Business"}.items():
        if k in text:
            return v
    return "Other"

# ----- prompts -----
classification_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a precise news topic classifier. You reply
with ONE word only: "
        "Business, Entertainment, Politics, Sport or Tech. No
explanation."),
    ("human", "Article:\nShares in the bank rose 4% after it reported
record annual "
        "pre-tax profits.\nCategory:"),
    ("ai", "Business"),
    ("human", "Article:\nThe actress collected the best supporting
role award at Sunday's "
        "ceremony.\nCategory:"),
    ("ai", "Entertainment"),
    ("human", "Article:\nThe home secretary defended the new anti-
terror bill in the "
        "Commons.\nCategory:"),
    ("ai", "Politics"),
    ("human", "Article:\nThe defending champion eased through to the
quarter-finals in "
        "straight sets.\nCategory:"),
    ("ai", "Sport"),
    ("human", "Article:\nThe firm released a security patch for its
browser after "
        "researchers found a flaw.\nCategory:"),
    ("ai", "Tech"),
    ("human", "Analyze the following news article and identify its
topic as one of the "
        "following categories: Business, Entertainment,
Politics, Sport, or Tech.\n\n"
        "Title: {title}\nArticle:\n{article}\n\nCategory:"),
])

summary_prompt = ChatPromptTemplate.from_messages([

```

```

        ("system", "You are a professional news editor. You write factual,
neutral summaries. "
        "Never add opinion. Never start with 'This article' or
'Here is a summary'."),
        ("human", "Summarize the main points of the following news article
in 2-3 sentences. "
        "Cover who, what, when, where and why where applicable.\n\n"
        "Title: {title}\nArticle:\n{article}\n\nSummary:"),
    ])

class Entities(BaseModel):
    people: List[str] = Field(default_factory=list)
    organizations: List[str] = Field(default_factory=list)
    locations: List[str] = Field(default_factory=list)

entity_parser = JsonOutputParser(pydantic_object=Entities)

entity_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert named-entity extraction system.
Output ONLY valid JSON – "
    "no markdown fences, no commentary. Do not invent
entities."),
    ("human", "From the article below, list the names of any important
people, organizations "
    "and places mentioned.\n\n{format_instructions}\n\n"
    "Title: {title}\nArticle:\n{article}\n\nJSON:"),
]).partial(format_instructions=entity_parser.get_format_instructions())

# ----- entity parsing helpers -----
def parse_entities(raw: str) -> dict:
    empty = {"people": [], "organizations": [], "locations": []}
    if not raw:
        return empty
    text = re.sub(r"<think>.*?</think>", "", raw, flags=re.DOTALL)
    text = re.sub(r"```(?:json)?|```", "", text).strip()
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        return empty
    try:
        data = json.loads(match.group(0))
    except json.JSONDecodeError:
        return empty
    out = {}
    for key in empty:
        val = data.get(key, [])
        if isinstance(val, str):
            val = [v.strip() for v in val.split(",") if v.strip()]
        out[key] = [str(v).strip() for v in val if str(v).strip()]

```

```

    return out

def flatten_entities(ent: dict) -> list:
    seen, flat = set(), []
    for v in ent["people"] + ent["organizations"] + ent["locations"]:
        if v.lower() not in seen:
            seen.add(v.lower())
            flat.append(v)
    return flat

try:
    print("Smoke test :", llm.invoke("Reply with exactly one word: OK").content.strip()[:40])
except Exception as e:
    msg = str(e)
    if "refused" in msg.lower() or "connection" in msg.lower():
        print("Ollama server not reachable – start it with: ollama serve")
    elif "not found" in msg.lower():
        print(f"Model missing – pull it with: ollama pull {OLLAMA_MODEL}")
    else:
        print(f"{type(e).__name__}: {msg[:300]}")

Model missing – pull it with: ollama pull llama3.2:3b

```

Bonus (optional, +10 marks): Run on ALL 2,225 articles

Use **Ollama** for this (set `LLM_PROVIDER = "ollama"` at the top and restart) — Groq's free tier will rate-limit long before $2,225 \times 3$ calls. This cell checkpoints to disk every `SAVE EVERY` articles, so an interrupted run can be resumed instead of restarted.

```

# =====
# BONUS: run the pipeline on ALL 2,225 articles (Ollama, checkpointed)
# =====
import os, re, json, time
import pandas as pd
from tqdm.auto import tqdm
from langchain_ollama import ChatOllama
from langchain_core.output_parsers import StrOutputParser

RUN_FULL_DATASET = True
CHECKPOINT = "part1_full_checkpoint.jsonl"
SAVE EVERY = 25
TEXT_LIMIT = 6000
MAX_RETRIES = 3
SLEEP = 0.0 # no rate limits locally

```

```

OLLAMA_MODEL      = "llama3.2:3b" # ollama pull llama3.2:3b
VALID_TOPICS      = set(CATEGORIES)

# ----- one client, all three chains rebuilt against it
-----
llm = ChatOllama(model=OLLAMA_MODEL, temperature=0, num_predict=512)

classification_chain = classification_prompt | llm | StrOutputParser()
summarization_chain  = summary_prompt       | llm | StrOutputParser()
entity_chain         = entity_prompt        | llm | StrOutputParser()

print("Using model:", OLLAMA_MODEL)
print("Smoke test :", llm.invoke("Reply with exactly one word: OK").content.strip()[:40])

# ----- helpers -----
def safe_invoke(chain, payload, default=""):
    for attempt in range(MAX_RETRIES):
        try:
            return chain.invoke(payload)
        except Exception as e:
            msg = str(e)
            if "connection" in msg.lower() or "refused" in msg.lower():
                raise RuntimeError("Ollama server not reachable – run `ollama serve`")
            wait = 2 ** attempt
            print(f" ! {type(e).__name__} – {msg[:120]} – retry in {wait}s")
            time.sleep(wait)
    return default

def strip_reasoning(text) -> str:
    if not isinstance(text, str):
        text = getattr(text, "content", "") or str(text)
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL)
    return re.sub(r"^````(?:\w+)?|````$", "", text.strip(), flags=re.MULTILINE).strip()

def process_article(row) -> dict:
    payload = {"title": row["Title"], "article": row["Article_Text"][:TEXT_LIMIT]}
    topic_raw = safe_invoke(classification_chain, payload)
    summary    = strip_reasoning(safe_invoke(summarization_chain, payload))
    entities   = parse_entities(safe_invoke(entity_chain, payload))
    return {
        "Article_ID":      int(row["Article_ID"]),

```

```

        "Detected_Topic": normalise_category(topic_raw),
        "Summary":        summary,
        "Key_Entities":   flatten_entities(entities),
        "People":         entities["people"],
        "Organizations":  entities["organizations"],
        "Locations":      entities["locations"],
    }

# ----- the run -----
if RUN_FULL_DATASET:
    done = set()
    if os.path.exists(CHECKPOINT):
        with open(CHECKPOINT, encoding="utf-8") as f:
            for line in f:
                if not line.strip():
                    continue
                try:
                    record = json.loads(line)
                    topic = record.get("Detected_Topic")
                    summary = str(record.get("Summary", "")).strip()
                    if topic in VALID_TOPICS and summary:
                        done.add(int(record["Article_ID"]))
                except (json.JSONDecodeError, KeyError, TypeError,
ValueError):
                    print("Skipping malformed checkpoint record")
            print(f"Resuming - {len(done)} complete articles already
processed")

    df_full["Article_ID"] = df_full["Article_ID"].astype(int)
    todo = df_full[~df_full["Article_ID"].isin(done)]
    print(f"Total: {len(df_full)} | already complete: {len(done)} | to
process now: {len(todo)}")

    buffer = []
    with open(CHECKPOINT, "a", encoding="utf-8") as f:
        for i, (_, row) in enumerate(tqdm(todo.iterrows(),
total=len(todo),
                                desc="Full dataset"), 1):
            buffer.append(process_article(row))
            if SLEEP:
                time.sleep(SLEEP)
            if i % SAVE_EVERY == 0 or i == len(todo):
                for r in buffer:
                    f.write(json.dumps(r, ensure_ascii=False) + "\n")
                f.flush()
                buffer = []

# ----- merge, dropping failed rows -----
all_results = pd.read_json(CHECKPOINT, lines=True)
all_results["Article_ID"] = all_results["Article_ID"].astype(int)

```

```

all_results = all_results[
    all_results["Detected_Topic"].isin(VALID_TOPICS)
    & all_results["Summary"].astype(str).str.strip().ne("")
]
all_results = all_results.drop_duplicates(subset="Article_ID",
keep="last")

full_final = df_full.merge(all_results, on="Article_ID",
how="left")
missing = full_final["Detected_Topic"].isna().sum()
print(f"Merged {len(all_results)} good results | rows still
missing output: {missing}")

full_final.to_csv("part1_bbc_results_full.csv", index=False)
print("Saved: part1_bbc_results_full.csv")

scored = full_final.dropna(subset=["Detected_Topic"])
acc_full = (scored["Detected_Topic"].str.lower()
== scored["True_Category"].str.lower()).mean()
print(f"\nFull-dataset accuracy: {acc_full:.1%} ({len(scored)}
articles scored)")
display(pd.crosstab(scored["True_Category"],
scored["Detected_Topic"]))
else:
    print("Set RUN_FULL_DATASET = True to attempt the bonus.")

```

Using model: llama3.2:3b

Smoke test : OK

Resuming – 75 complete articles already processed

Total: 2225 | already complete: 75 | to process now: 2150

```

{"model_id": "38830727d3ab4b63ada3ef2b6594f39c", "version_major": 2, "vers
ion_minor": 0}

```

Merged 2225 good results | rows still missing output: 0

Saved: part1_bbc_results_full.csv

Full-dataset accuracy: 83.1% (2225 articles scored)

Detected_Topic	Business	Entertainment	Politics	Sport	Tech
True_Category					
business	422	0	81	0	7
entertainment	48	309	29	0	0
politics	6	0	411	0	0
sport	9	1	53	448	0
tech	120	13	8	1	259